ЛАБОРАТОРНА РОБОТА № 1

Тема:
Асинхронна обробка даних у TypeScript

Мета:
Метою лабораторної роботи є налаштування середовища розробки та набуття практичних навичок асинхронного програмування в TypeScript, зокрема: • встановлення Node.js та налаштування TypeScript-проєкту; • використання Promise.all та Promise.race для керування паралельними операціями; • реалізація механізмів повторних спроб (retry) при помилках; • послідовна обробка даних партіями (batch processing); • коректна обробка помилок через try/catch в асинхронному коді.

Хід роботи:

Завдання №3.2

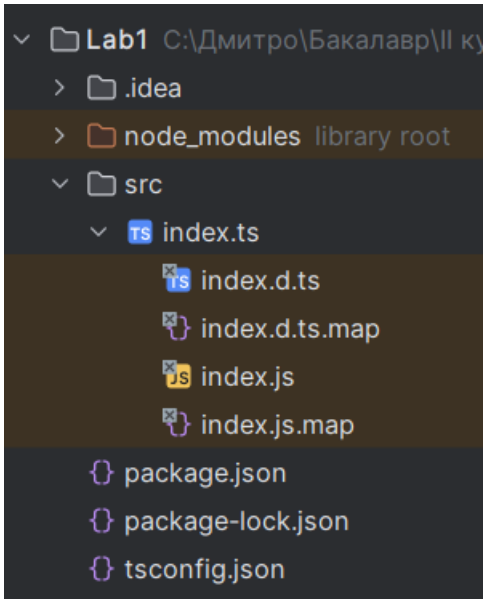

Рис.1 Структура проекту

Завдання №3.3
Функція delay
Створює Promise, що завершується через вказаний час. Використовується як допоміжна функція в інших завданнях. Вимоги: • Приймає кількість мілісекунд затримки. • Повертає Promise, що завершується через ms мілісекунд. • Використати конструктор new Promise та setTimeout.

Лістинг програми функція deley:

```
function delay(ms: number) : Promise<void>{
    return new Promise((resolve) => {
        setTimeout(resolve, ms);
    });
}
```

```
Функція delay

Початок програми.
Готово затримка завершилась
```

Рис.2 Результат виконання функції

Завдання №3.4

Функція fetchUserProfiles

Симулює паралельне завантаження профілів користувачів з сервера. Вимоги: • Кожен запит має симулювати затримку 50–150 мс (випадково). • Використати Promise.all для паралельного завантаження. • Якщо userIds порожній — повернути порожній масив. • Всі змінні та параметри мають бути типізовані.

Лістинг програми функція fetchUserProfiles:

```
function fetchUserProfiles(userIds: string[]) : Promise<UserProfile[]> {

    if(userIds.length === 0){
        return Promise.resolve([]);
    }
    const promises = userIds.map(id => {
        return new Promise<UserProfile>((resolve) => {
            let min: number = 50;
            let max: number = 150;
            const delayTime = Math.floor(Math.random() * (max - min + 1) +
min);

            setTimeout(() =>{
                resolve({
                    id: id,
                    name: `User ${id}`,
                    email: `${id}@example.com`,
                });
            }, delayTime);
        });
    });
    return Promise.all(promises);
}
```

```
Старт завантаження
[
  { id: 'user1', name: 'User user1', email: 'user1@example.com' },
  { id: 'user2', name: 'User user2', email: 'user2@example.com' },
  { id: 'user3', name: 'User user3', email: 'user3@example.com' }
]
```

Рис.3 Результат виконання функції

Завдання №3.5

Функція retryOperation

Виконує асинхронну операцію з автоматичними повторними спробами при помилці. Вимоги: • При помилці — зачекати 100 мс, спробувати знову. • Максимум maxRetries спроб (за замовчуванням 3). • Якщо всі спроби провалилися — викинути останню помилку. • Логувати кожну спробу: console.log('Спроба ${n}...').

Лістинг програми функція retryOperation:

```
async function retryOperation<T>(
    operation: () => Promise<T>,
    maxRetries: number = 3
): Promise<T> {
    for (let i = 0; i < maxRetries; i++) {
        console.log(`спроба ${i}`);
        try{
            return await operation();
        }catch (error){
            console.log("Помилка. Чекаємо...");
            await delay(100);
        }
    }
    throw new Error("Всі спроби вичерпано");
}
```

```
Функція retryOperation

спроба 0
Помилка. Чекаємо...
спроба 1
Помилка. Чекаємо...
спроба 2
Результат: Успіх
```

Рис.4 Результат виконання функції

Завдання №3.6

Функція processInBatches

Обробляє великий масив елементів партіями (batches) послідовно. Вимоги: • Розділити items на партії розміром batchSize. • Обробити партії послідовно (не паралельно!). • Кожну партію обробити функцією processor. • Повернути масив усіх результатів. • Логувати прогрес: console.log('Обробка партії ${n}/${total}...').

Лістинг програми функція processInBatches:

```
async function processInBatches(
    items: number[],
    batchSize: number,
    processor: (batch: number[]) => Promise<number[]>
): Promise<number[]> {
    const results: number[] = [];
    const totalBatches = Math.ceil(items.length / batchSize);
    let currentBatchNum = 0;
```

```
    for (let i = 0; i < items.length; i += batchSize) {
        currentBatchNum++;

        const batch = items.slice(i, i + batchSize);

        console.log(`Обробка партії ${currentBatchNum}/${totalBatches}`);

        const processedBatch = await processor(batch);
        results.push(...processedBatch);
    }
    return results;
}
```

```
Функція processInBatches

Обробка партії 1/3
Обробка партії 2/3
Обробка партії 3/3
Результат:  [
   2,  4,  6,  8,
  10, 12, 14, 16
]
```

Рис.5 Результат виконання функції

Завдання №3.7

Функція raceWithTimeout

Виконує Promise з обмеженням часу виконання. 9 Вимоги: • Використати Promise.race. • Якщо promise не завершився за timeoutMs — викинути помилку. • Текст помилки: 'Operation timed out after ${timeoutMs}ms'.

Лістинг програми функція raceWithTimeout:

```
function raceWithTimeout<T>(promise: Promise<T>, timeoutMs: number): Promise<T>
{
    const timeoutPromise = new Promise<never>((_, reject) => {
        setTimeout(() => {
            reject( new Error(`Operation timed out after ${timeoutMs}ms`));
        }, timeoutMs);
    });

    return Promise.race([promise, timeoutPromise]);
}
```

```
Функція raceWithTimeout
Результат1:  встиг
Помилка2:  Operation timed out after 100ms
Час вийшов Operation timed out after 100ms
```

Рис.6 Результат виконання функції

лістинг програми функція main(Виклик всіх функцій):

```
async function main(){
    console.log("Функція delay\n")
    console.log("Початок програми.");
    await delay(1000);
    console.log("Готово затримка завершилась");


    console.log("\nФункція fetchUserProfiles\n")
    console.log("Старт завантаження");
    const users = await fetchUserProfiles(["user1", "user2", "user3"]);
    console.log(users);

    console.log("\nФункція retryOperation\n");
    let attempts = 0;

    const unreliableOperation = async (): Promise<string> => {
        attempts++;
        if (attempts < 3) {
            throw new Error("Тимчасова помилка");
        }
        return "Успіх";
    };

    try{
        const result = await retryOperation(unreliableOperation, 3);
        console.log(`Результат: ${result}`);
    }catch (error){
        console.error("Операція провалилась остаточно");
    }

    console.log("\nФункція processInBatches\n");

    const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

    const  myProcessor = async (batch: number[]): Promise<number[]> => {
        await delay(1000);
        return batch.map(n => n * 2);
    };

    const result = await processInBatches(numbers, 3, myProcessor);
    console.log('Результат: ', result);

    console.log("\nФункція raceWithTimeout\n");

    const fast = delay(50).then(() => "встиг");
    try{
        const result = await raceWithTimeout(fast, 100);
        console.log("Результат1: ", result);
    }
    catch(error){
        console.log("Помилка1: ", error);
    }

    const slow = delay(200).then(() => "Повільний");
    try{
        await raceWithTimeout(slow, 100);
    }catch (error){
        if(error instanceof Error){
            console.error("Помилка2: ", error.message);
            console.log("Час вийшов", error.message);
        }
    }
}
```

Висновок: В даній лабораторній я здійснив налаштування середовища розробки та набув практичних навичок асинхронного програмування в TypeScript, зокрема: • встановлення Node.js та налаштування TypeScript-проєкту; • використання Promise.all та Promise.race для керування паралельними операціями; • реалізація механізмів повторних спроб (retry) при помилках; • послідовна обробка даних партіями (batch processing); • коректна обробка помилок через try/catch в асинхронному коді.